ChargeGrid Intelligence — Sprint 3: Prototipagem Funcional e Integração

Faculdade de Informática e Administração Paulista — Ciência da Computação — 1CCPH, Equipe 1

Challenge Sprint: Pensamento Computacional e Automação com Python (GoodWe Challenge)

Equipe

Integrante	RM
[PREENCHER: nome completo]	[PREENCHER]
[PREENCHER: nome completo]	[PREENCHER]
[PREENCHER: nome completo]	[PREENCHER]
[PREENCHER: nome completo]	[PREENCHER]

Links

- Repositório: https://github.com/guiosantos-dev/sprint2equipe1.1ccph
- Vídeo técnico (YouTube, não listado): [PREENCHER após gravar]

1. Resumo

O ChargeGrid Intelligence é o "cérebro" de um eletroposto comercial: ele decide, a cada 5 minutos, quanta potência cada carregador de veículo elétrico (EV) pode entregar, de modo que a instalação não ultrapasse o limite de 100 kW da rede, aproveitando a energia solar e a bateria.

Na Sprint 1 analisamos o problema (controle de demanda, protocolos abertos, tarifação e IA). Na Sprint 2 fizemos uma prova de conceito com interface Tkinter. Nesta Sprint 3 o protótipo passa a integrar **todos** os componentes em uma única simulação de um dia inteiro: geração solar, carga do prédio, bateria, seis carregadores de três marcas, mensagens no formato OCPP, balanceamento de carga, tarifação por horário e coleta de dados em arquivos.

Resultado principal (cenário simulado, seção 6): sem controle, a rede chega a **146,0 kW** e fica **80 minutos acima do limite**; com o ChargeGrid o pico cai para **98,2 kW**, sem nenhum minuto acima do limite, entregando a mesma energia aos veículos (371 kWh). O custo é um tempo de recarga médio maior (88,2 → 95,4 min).

O que este protótipo é — e o que não é

- **É uma simulação em Python.** Não há carregador, inversor ou bateria físicos. Solar, carga do prédio e veículos são gerados por código, com semente fixa (resultados reprodutíveis).
- **Todos os parâmetros são assumidos** (potências, tarifas, capacidade da bateria) e não representam especificações oficiais da GoodWe, ABB ou WEG. Os nomes de marca servem apenas para demonstrar interoperabilidade.
- **O "OCPP" é só o formato das mensagens** (JSON no padrão `[2, id, ação, payload]`) gravadas em arquivo. Não há conexão WebSocket nem certificação com o protocolo.
- **A "IA" é uma previsão por regressão linear, opcional e desligada por padrão.** Nos testes (seção 6.4) ela **não** melhorou a proteção contra sobrecarga. O que evita a sobrecarga é a margem de segurança de 5% e o balanceamento.

2. Arquitetura e integração dos componentes

Arquitetura do protótipo ChargeGrid Intelligence (Sprint 3)

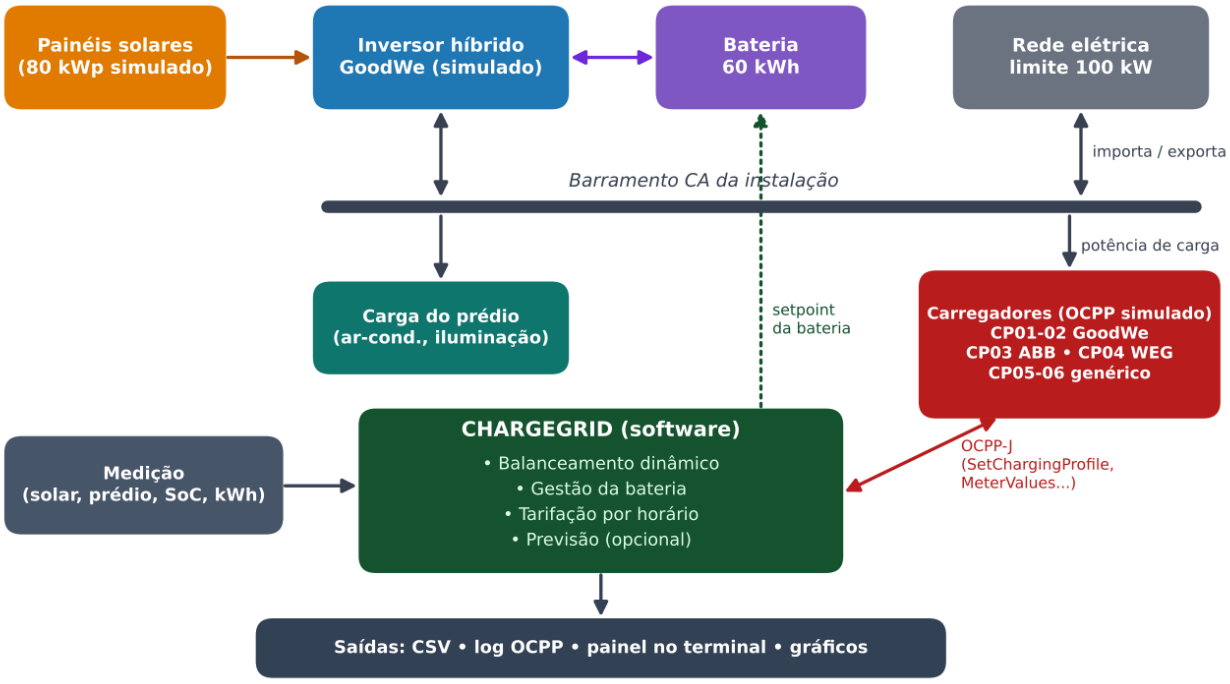
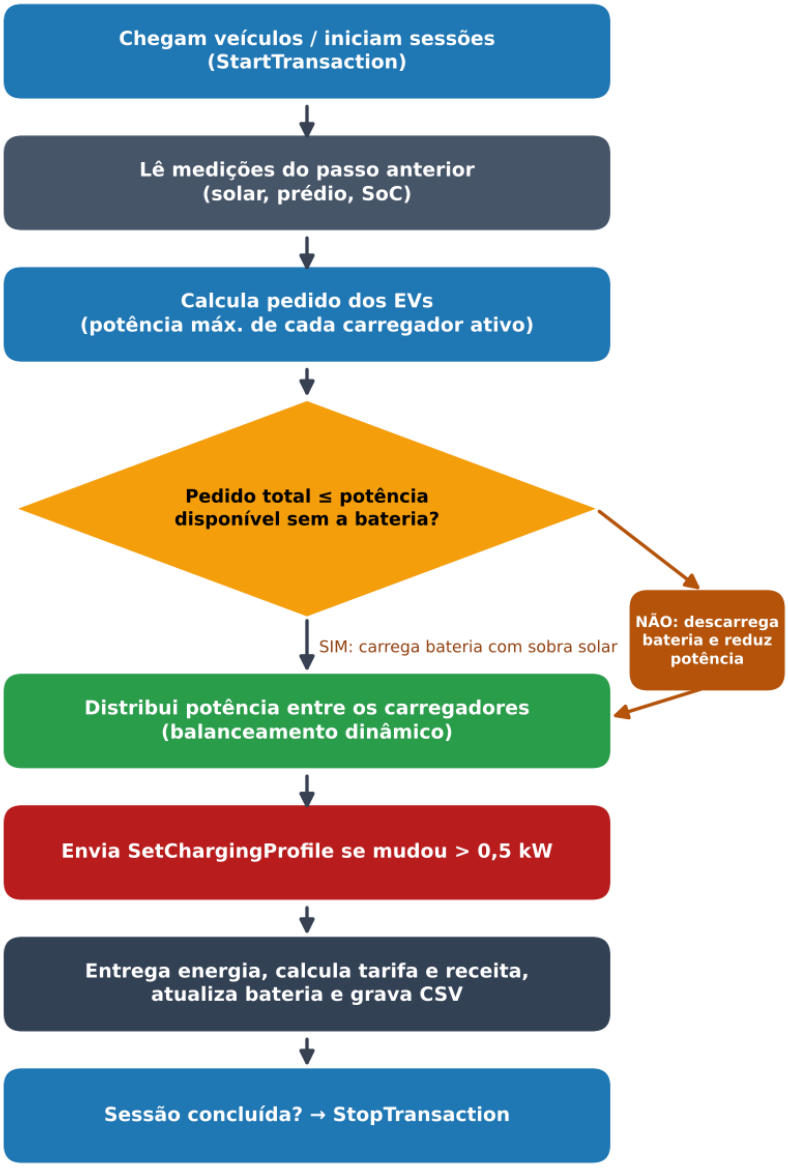


Diagrama de blocos

Fluxo de energia: painéis solares → inversor híbrido ↔ bateria → barramento CA, que também conecta a rede elétrica, a carga do prédio e os carregadores. Fluxo de informação: medições → ChargeGrid → comandos para a bateria e `SetChargingProfile` (OCPP simulado) para cada carregador.

Fluxograma do ciclo de decisão (a cada 5 minutos simulados)



Fluxograma de decisão

A cada passo o ChargeGrid soma o que os veículos pedem e compara com a potência disponível sem usar a bateria (`limite × 0,95 + solar − prédio`). Se cabe, atende tudo e guarda a sobra solar na bateria. Se não cabe, descarrega a bateria; se ainda faltar, divide a potência restante entre os carregadores ativos.

3. Justificativa técnica das escolhas

Escolha	Por quê
Python puro (biblioteca padrão) para o motor	Reprodutível em qualquer máquina, sem dependências; combina com a disciplina. Só os gráficos usam `matplotlib`.
Simulação com passo de 5 min e semente fixa	Dá 288 pontos por dia, suficiente para ver picos e rampas; a semente permite repetir o experimento e comparar modos com o mesmo "dia".
Balanceamento por divisão igualitária com redistribuição da sobra (*water-filling*)	Algoritmo simples, justo e fácil de explicar: quem pede menos fica satisfeito e a sobra é repartida entre os demais.
Margem de segurança de 5%	O controlador decide com a medição do passo anterior; a margem absorve variações entre uma medição e a seguinte. Sem ela (margem 1,0) houve sobrecarga em 30 de 30 execuções (seção 6.4).
Bateria só carrega com sobra solar	Garante que a energia usada na ponta é de origem solar e evita comprar energia cara para guardá-la.
Tarifa por horário (R$ 0,90 solar 10–15h, R$ 1,20 base, R$ 1,80 ponta 18–21h)	Mostra a tarifação dinâmica do pilar da Sprint 1. Valores ilustrativos; a base de R$ 1,20 é a mesma da Sprint 2.
Mensagens no formato OCPP-J	Representa o pilar "protocolos abertos": carregadores de marcas diferentes falam a mesma língua com a central.
Saída em CSV e JSONL	Formatos abertos, fáceis de abrir em planilha ou ler por outro programa.
Previsão linear mantida como opção	Implementada e medida; como não trouxe ganho, fica desligada por padrão.

4. Componentes simulados

Componente	Parâmetros assumidos
Painéis solares	pico de 80 kW, curva de sol entre 6h e 18h com nuvens aleatórias
Inversor híbrido (estilo GoodWe)	integra solar, bateria e barramento
Bateria	60 kWh, ±30 kW, SoC entre 10% e 95%, começa em 20%, eficiência de 95% em cada sentido
Rede elétrica	limite de 100 kW (mesmo valor da Sprint 2)
Carga do prédio	25 kW de base, +15 kW em horário comercial, +10 kW das 18h às 20h, ruído de ±2 kW
Carregadores	CP01 e CP02 GoodWe (22 kW), CP03 ABB (22 kW), CP04 WEG (22 kW), CP05 e CP06 genéricos (11 kW)
Sessões de recarga	14 veículos entre 9h e 18h45, de 15 a 40 kWh cada (tabela completa em `data/chargegrid_sessoes.csv`)

5. Como o ChargeGrid atua — exemplo real das 18:45

Nesse instante há seis veículos conectados pedindo 110 kW. A bateria já chegou ao mínimo (10%, às 18:40) e não há sol. O ChargeGrid limita cada carregador a 9,97 kW e envia o comando:

```
{"hora": "18:45", "de": "CENTRAL", "para": "CP01", "msg": [2, "00117", "SetChargingProfile",
  {"connectorId": 1, "limite_kW": 9.97, "motivo": "balanceamento (limite da rede)"}]}
```

Cada carregador responde com `MeterValues` (energia e potência da sessão). A rede fica em 95,8 kW, abaixo do limite. A imagem abaixo é a **saída real** do modo ao vivo (`python src/main.py --live`) renderizada como imagem:

```
=== 18:45 | tarifa R$ 1.80/kWh | decisão: balanceamento (limite da rede) ===
 Solar       0.0 kW  [............................]
 Prédio     36.0 kW  [##########..................]
 EVs        59.8 kW  [##################..........] (pedido 110.0)
 Bateria    +0.0 kW  SoC  10.0%
 REDE       95.8 kW  [###################.........] limite 100 kW -> normal
   CP01 GoodWe          10.0/22 kW  sessão 11:  16.7/35 kWh
   CP02 GoodWe          10.0/22 kW  sessão 9:  27.7/32 kWh
   CP03 ABB             10.0/22 kW  sessão 14:   0.8/28 kWh
   CP04 WEG             10.0/22 kW  sessão 10:  22.2/30 kWh
   CP05 Genérico OCPP   10.0/11 kW  sessão 12:   6.3/20 kWh
   CP06 Genérico OCPP   10.0/11 kW  sessão 13:   3.6/15 kWh
```

Terminal ao vivo 18:45

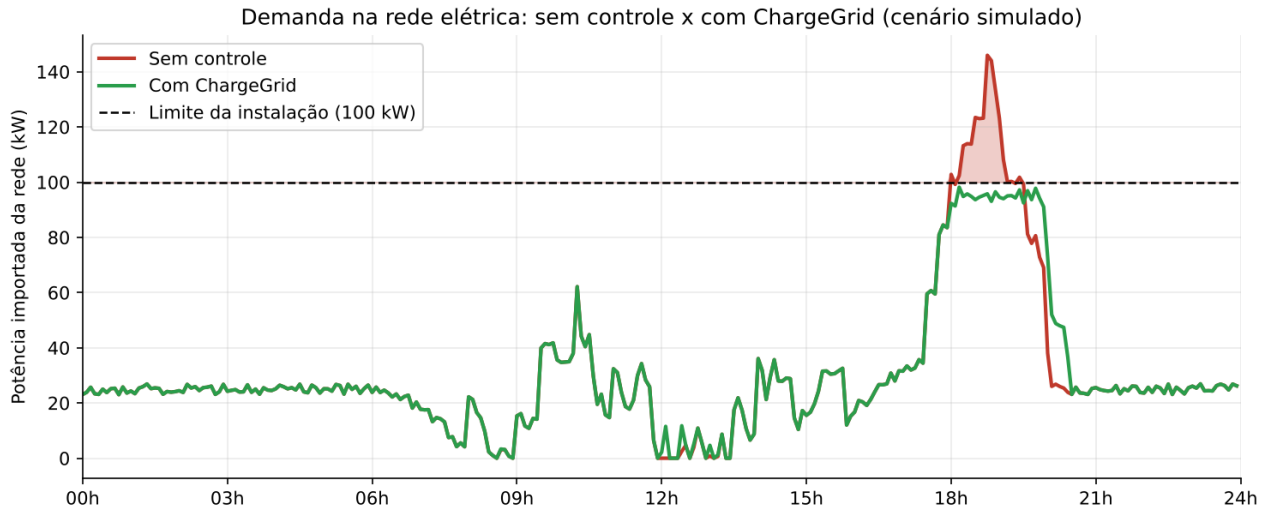
6. Resultados e dados funcionais

Todos os números vêm de `python src/main.py` e estão em `data/`. O mesmo "dia" simulado foi usado nos dois modos.

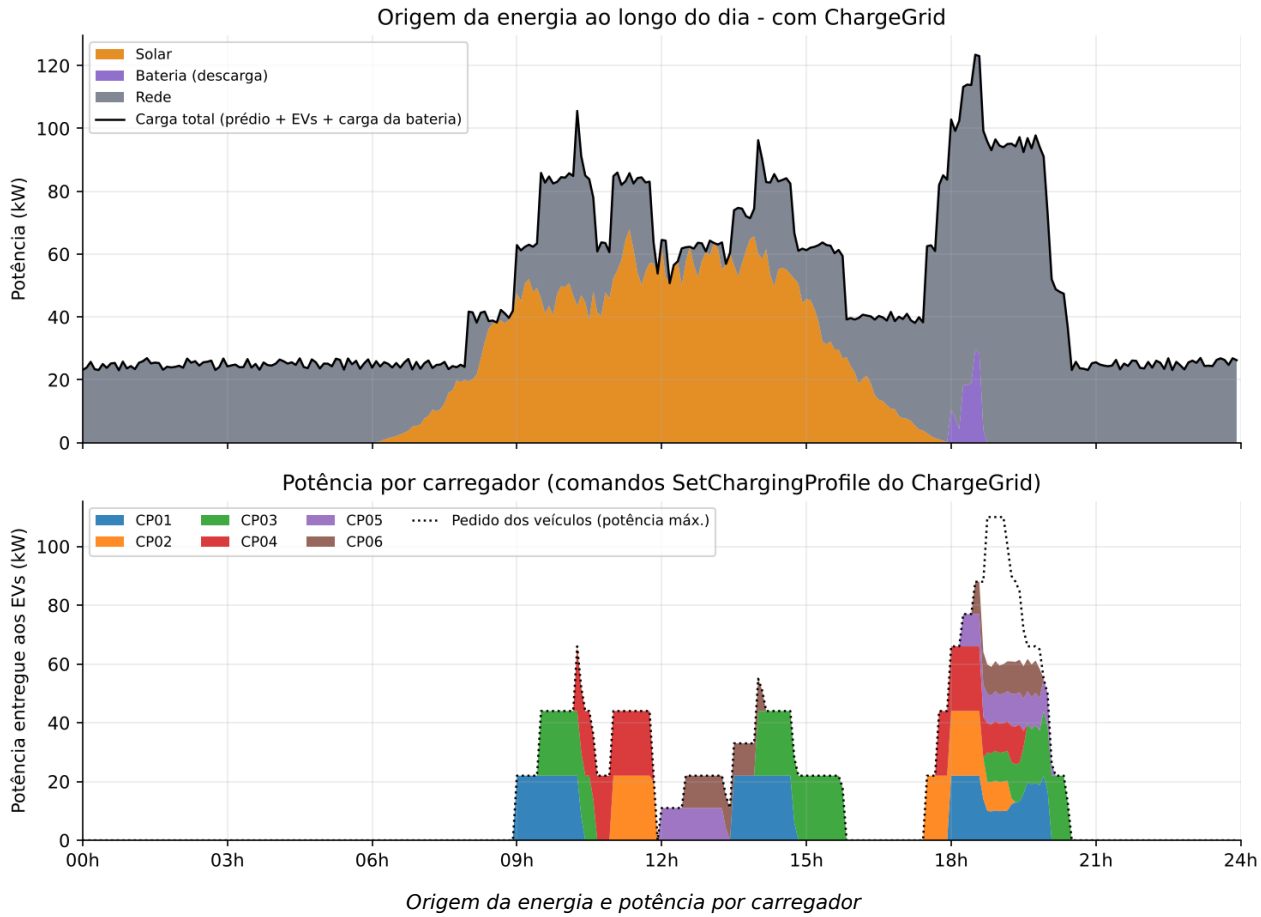
6.1 Sem controle x com ChargeGrid

Indicador	Sem controle	Com ChargeGrid
Pico de potência na rede	146,0 kW	98,2 kW
Minutos acima do limite de 100 kW	80	0
Energia acima do limite	22,73 kWh	0 kWh
Energia entregue aos veículos	371,0 kWh	371,0 kWh
Sessões concluídas	14 de 14	14 de 14
Duração média da recarga	88,2 min	95,4 min
Receita simulada	R$ 483,30	R$ 483,30
Energia comprada da rede	724,9 kWh	715,6 kWh
Parcela renovável do consumo	36,5%	37,3%
Mensagens OCPP simuladas	122	185 (56 são `SetChargingProfile`)

A receita é igual porque todas as sessões terminam antes das 21h nos dois modos, ou seja, na mesma faixa de tarifa. A "parcela renovável" considera todo o consumo (prédio + veículos), não só os EVs.



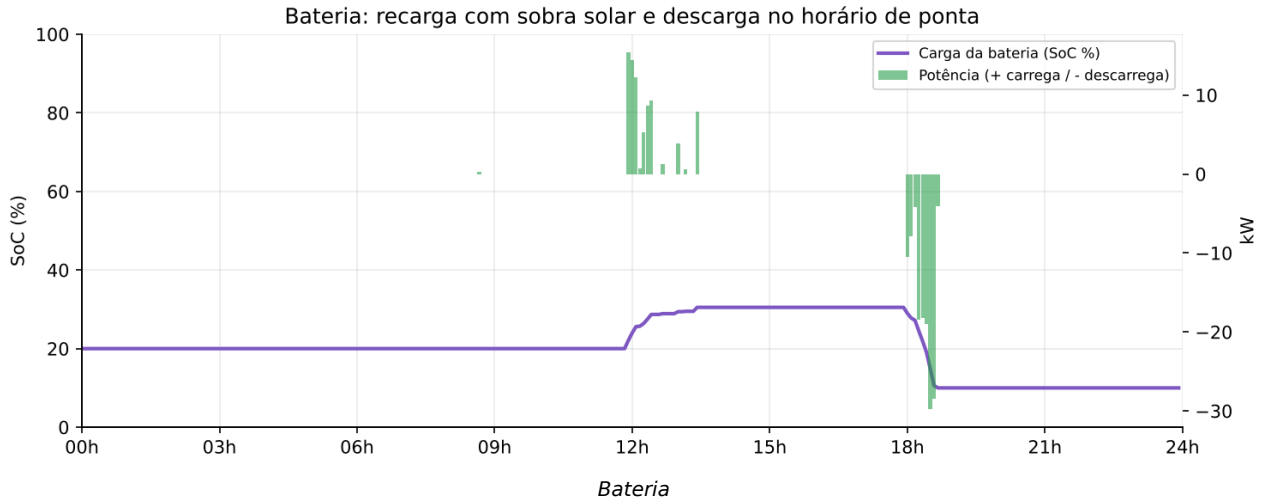
Demanda na rede



Origem da energia e potência por carregador

6.2 Bateria

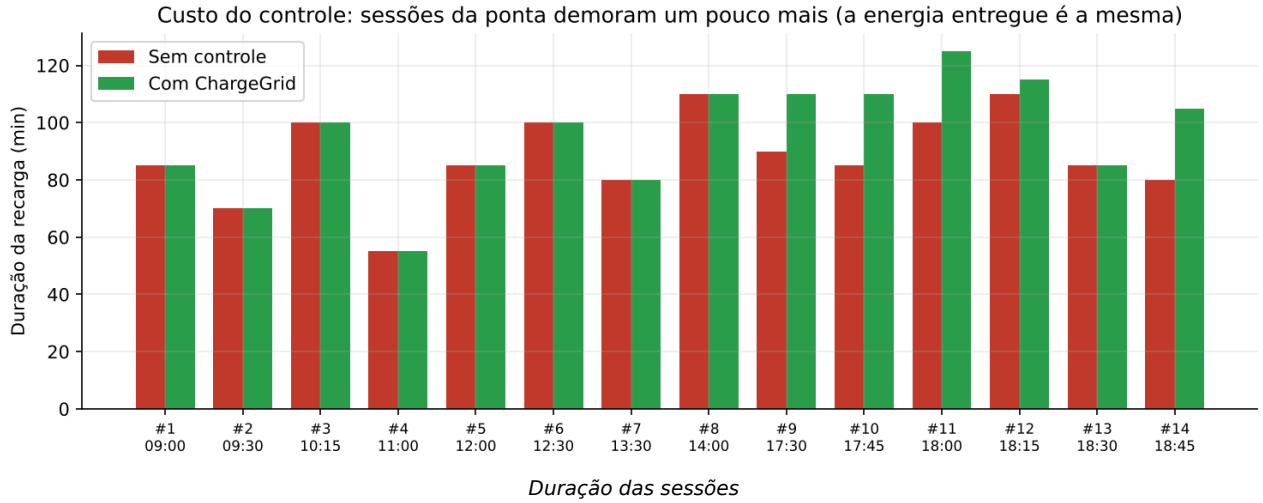
A bateria carregou **6,65 kWh** com sobra solar por volta do meio-dia (SoC de 20% para 30,5%) e descarregou **11,7 kWh** na ponta, esgotando-se às 18:40. Ou seja, **a bateria sozinha não cobre toda a ponta** neste cenário: depois que ela acaba, quem protege a rede é o balanceamento. A sobra solar é pequena porque prédio e veículos consomem quase toda a geração; apenas 3,07 kWh foram exportados.



Bateria

6.3 Custo do controle

Limitar a potência faz as recargas na ponta demorarem mais (até cerca de 25 min a mais nas sessões 9, 10, 11 e 14); a energia entregue é a mesma e nenhuma sessão ficou sem concluir.



6.4 A previsão por regressão linear ajudou? Não

Rodamos 30 "dias" com sementes diferentes (`python src/analise_sensibilidade.py`, resultado em `data/ablacao_previsao.csv`):

Margem de segurança	Previsão	Pico médio na rede	Minutos médios acima do limite	Execuções com sobrecarga	Mensagens `SetChargingProfile` (média)
0,95	não	97,88 kW	0,00	0 de 30	52,1
0,95	sim	98,72 kW	0,17	1 de 30	46,7
1,00	não	102,80 kW	48,83	30 de 30	45,0
1,00	sim	103,71 kW	53,50	30 de 30	41,1

Conclusões honestas: (1) a **margem de segurança** é o que evita a sobrecarga; (2) a previsão linear não reduziu a sobrecarga e ficou um pouco pior, mas enviou cerca de 10% menos comandos. Por isso o padrão é o controlador reativo. Um modelo de IA mais elaborado (como proposto na Sprint 1) fica como trabalho futuro.

7. Contribuição de cada tecnologia

Tecnologia	Sustentabilidade	Automação inteligente	Eficiência energética
Solar + inversor híbrido	Parte do consumo vem de fonte renovável (36,5–37,3% do consumo do dia no cenário)	Inversor e controlador compartilham medições	98,3–99,3% da geração solar foi aproveitada
Bateria	Energia da ponta vem de sobra solar guardada (11,7 kWh)	Carrega e descarrega sozinha conforme a demanda	Reduz o pico comprado da rede
Balanceamento dinâmico	Evita ampliar a infraestrutura elétrica só para a ponta	Recalcula a potência de cada carregador a cada 5 min	Rede de 146,0 kW para 98,2 kW de pico
OCPP simulado	Permite misturar marcas sem trocar equipamento	56 comandos `SetChargingProfile` sem intervenção humana	Mensagens `MeterValues` dão visibilidade do consumo
Tarifação por horário	Incentiva recarregar quando há sol	Cálculo automático por sessão	Sinaliza ao usuário o horário de ponta
Registro em CSV/JSONL	Rastreabilidade dos dados	Coleta automática, sem digitação	Base para análises futuras

8. Conexão com a disciplina (Pensamento Computacional e Automação com Python)

- **Decomposição:** o problema foi dividido em módulos (`config`, `simulacao`, `controlador`, `ocpp_sim`, `gerar_graficos`).
- **Abstração:** solar, bateria e carregadores são representados só pelos números que importam (kW, kWh, SoC).
- **Reconhecimento de padrões:** a ponta das 18h aparece nos dados; a regressão linear tenta extrapolar a tendência.
- **Algoritmos:** balanceamento por *water-filling* e a regra de decisão do fluxograma.
- **Automação com Python:** laço de simulação, geração de CSV/JSON, gráficos e comandos automatizados, tudo sem intervenção manual.
- **Estruturas de dados e arquivos:** listas, dicionários, `dataclass`, leitura e escrita de CSV e JSON Lines.

9. Limitações

- Tudo é simulado; não foi testado com hardware, protocolo OCPP real ou dados reais de uma instalação.
- Um único cenário de um dia (mais 30 variações de ruído só na análise da seção 6.4); a bateria carregou pouco porque a sobra solar é pequena.
- Os veículos pedem sempre a potência máxima do carregador (sem curva de carga real de bateria).
- O controlador decide com a medição do passo anterior; não há modelo de falha de internet (a operação híbrida online/offline da Sprint 1 não foi implementada).
- A interface gráfica Tkinter da Sprint 2 foi mantida em `sprint2_pcap/`, mas **não** foi integrada nem retestada nesta sprint; a demonstração da Sprint 3 é feita no terminal.

10. Como executar

Requisitos: Python 3 (testado com Python 3.13; a simulação não precisa de bibliotecas externas). Para os gráficos: `pip install -r requirements.txt`.

```
python src/main.py                    # roda os dois cenários e grava os arquivos em data/
python src/main.py --live             # demonstração ao vivo no terminal (17:00 às 21:00)
python src/main.py --live --extra     # com um veículo extra conectado às 18:00
python src/main.py --live --velocidade 0.15 --de 17:30 --ate 20:30
python src/main.py --previsao         # liga a previsão linear (experimental)
python src/analise_sensibilidade.py   # análise com 30 sementes (seção 6.4)
python src/gerar_graficos.py          # gráficos e diagramas em docs/
```

11. Estrutura do repositório

```
README.md                     este documento
requirements.txt              dependência dos gráficos (matplotlib)
src/
  config.py                   parâmetros assumidos do cenário
  simulacao.py                planta (solar, prédio, bateria, carregadores) e laço principal
  controlador.py              balanceamento, gestão da bateria e previsão
  ocpp_sim.py                 mensagens no formato OCPP-J (simuladas)
  main.py                     execução e demonstração ao vivo
  analise_sensibilidade.py    comparação com/sem previsão em 30 sementes
  gerar_graficos.py           gráficos e diagramas
data/                         CSVs, logs OCPP (JSONL) e métricas gerados pela simulação
docs/diagramas/               diagrama de blocos e fluxograma
docs/resultados/              gráficos e imagem da saída do terminal
docs/roteiro_video.md         roteiro do vídeo de até 5 minutos
sprint2_pcap/                 código da Sprint 2 (Tkinter)
```

12. Evolução do projeto

- **Sprint 1:** análise teórica (GoodWe, ChargeGrid, quatro pilares, problemas e soluções).
- **Sprint 2:** prova de conceito em Tkinter (3 carregadores, sobrecarga fixa, redistribuição proporcional e tarifa de R$ 1,20/kWh).

- **Sprint 3:** simulação integrada de um dia com solar, bateria, 6 carregadores, mensagens OCPP, tarifação por horário, coleta de dados e análise dos resultados.